

Series de tiempo de los puntajes factoriales y la excedencia

MA2003B, equipo 7

Tabla de contenidos

0. Carga y verificación de los nombres	1
1. La cobertura no es opcional: por qué agregar a área metropolitana	3
Dependencia transversal: la violación de independencia que no está documentada	5
2. Construcción de la serie de área metropolitana	6
3. Descomposición MSTL: periodos 7 (semanal) y 365 (anual)	7
4. ACF y PACF hasta el lag 40: serie cruda vs residual MSTL	13
5. Ljung-Box en lags 7, 14 y 30 (ambas versiones)	18
6. Rezagos: contra la fecha calendario, no <code>shift()</code> de fila	19
7. Criterios de aceptación	19
8. Qué queda para el reporte	19

09_extraccion_factorial.pdf dejó tres puntajes factoriales por día-estación (`af_puntajes_diarios.parquet`): combustión primaria, ventilación y forzamiento fotoquímico. Esta notebook no discute su origen ni su validez — eso ya se cerró— sino que caracteriza **cómo se comportan en el tiempo**, junto con `excede_anio_5` (el escenario principal de modelado, umbral MDA8 > 51 ppb) del dataset diario transformado (`sima_transformado_diario.parquet`).

El bloque hereda el recorte 2021–2024 de entrenamiento + 2025 de validación que fijó 09_extraccion_factorial.qmd (2020 queda fuera: solo tenía 13 de las 15 estaciones). No se reabre esa decisión aquí.

0. Carga y verificación de los nombres

```
import sys
from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from statsmodels.tsa.seasonal import MSTL
from statsmodels.tsa.stattools import acf, pacf
```

```

from statsmodels.stats.diagnostic import acorr_ljungbox
# No fijar matplotlib.use("Agg") aqui: Quarto necesita el backend interactivo
# para capturar las figuras de plt.show() (ver PR #50).

RAIZ = Path.cwd().parent
PROC = RAIZ / "data" / "processed"
FIGS = RAIZ / "reports" / "figuras"
FIGS.mkdir(parents=True, exist_ok=True)
sys.path.insert(0, str(RAIZ))

plt.rcParams.update({"font.size": 8, "font.family": "serif",
                    "axes.spines.top": False, "axes.spines.right": False,
                    "figure.dpi": 150})
AZUL, ROJO, GRIS, CLARO = "#3b6ea5", "#b5443a", "#8c8c8c", "#d9d9d9"

af = pd.read_parquet(PROC / "af_puntajes_diarios.parquet")
di = pd.read_parquet(PROC / "sima_transformado_diario.parquet")
print(f"puntajes factoriales: {af.shape[0]:,} filas x {af.shape[1]} columnas")
print(f"diario transformado: {di.shape[0]:,} filas x {di.shape[1]} columnas")

```

```

puntajes factoriales: 19,036 filas x 6 columnas
diario transformado: 26,691 filas x 34 columnas

```

af_puntajes_diarios.parquet guarda los puntajes como F1/F2/F3, en el mismo orden de columnas que NOMBRES en 09_extraccion_factorial.qmd (Combustión primaria, Ventilación, Forzamiento fotoquímico). No se renombra a ciegas: se verifica contra las cargas de af_cargas.csv, que dicen que NO2_06_09 carga en el primer factor, WSR_media en el segundo y TOUT_max en el tercero.

```

af = af.rename(columns={"F1": "combustion_primaria", "F2": "ventilacion",
                      "F3": "forzamiento_fotoquimico"})
FACTORES = ["combustion_primaria", "ventilacion", "forzamiento_fotoquimico"]

verifica = af.merge(di[["estacion", "fecha", "NO2_06_09", "WSR_media",
    ↪ "TOUT_max"]],
                  on=["estacion", "fecha"], how="inner")
chequeo = {
    "combustion_primaria vs NO2_06_09": verifica[["combustion_primaria",
    ↪ "NO2_06_09"]].corr().iloc[0, 1],
    "ventilacion vs WSR_media": verifica[["ventilacion",
    ↪ "WSR_media"]].corr().iloc[0, 1],
    "forzamiento_fotoquimico vs TOUT_max": verifica[["forzamiento_fotoquimico",
    ↪ "TOUT_max"]].corr().iloc[0, 1],
}
for nombre, r in chequeo.items():
    print(f" {nombre:42s} r = {r:.3f}")

```

```
assert all(r > 0.8 for r in chequeo.values()), "el renombrado F1/F2/F3 no
↳ corresponde a af_cargas.csv"
print("\nrenombrado F1/F2/F3 -> nombres de factor: verificado")
```

```
combustion_primaria vs NO2_06_09      r = 0.846
ventilacion vs WSR_media              r = 0.932
forzamiento_fotoquimico vs TOUT_max   r = 0.861
```

renombrado F1/F2/F3 -> nombres de factor: verificado

Las tres correlaciones superan 0.84: el renombrado es correcto, no una suposición sobre el orden de las columnas.

```
m = af.merge(di[["estacion", "fecha", "excede_anio_5"]], on=["estacion", "fecha"],
             how="inner").dropna(subset=["excede_anio_5"])
print(f"tabla dia-estacion (factores + excede_anio_5, ambos no nulos): {len(m):,}
↳ filas")
print(f"estaciones: {m.estacion.nunique()}   rango de fechas:
↳ {m.fecha.min().date()} a {m.fecha.max().date()}")
```

```
tabla dia-estacion (factores + excede_anio_5, ambos no nulos): 18,631 filas
estaciones: 15   rango de fechas: 2021-01-01 a 2025-12-31
```

1. La cobertura no es opcional: por qué agregar a área metropolitana

El calendario día-estación no es regular. Antes de tocar STL/MSTL —que exigen una serie sin huecos— hay que decidir cómo se llega a esa regularidad: agregar las 15 estaciones a un solo valor diario de área metropolitana, o filtrar a las estaciones con mejor cobertura y analizarlas por separado.

```
full_days = pd.date_range(m.fecha.min(), m.fecha.max(), freq="D")
n_est = m.estacion.nunique()
posible = len(full_days) * n_est
print(f"dias posibles: {len(full_days):,} dias x {n_est} estaciones = {posible:,}
↳ dia-estacion")
print(f"cobertura global: {100*len(m)/posible:.1f}%")

cobertura = (m.groupby("estacion").fecha.nunique() / len(full_days) *
↳ 100).sort_values()
print("\ncobertura por estacion (%, ascendente):")
print(cobertura.round(1).to_string())
print(f"\nestaciones con cobertura >= 85%: {(cobertura >= 85).sum()} de
↳ {len(cobertura)}")
```

dias posibles: 1,826 dias x 15 estaciones = 27,390 dia-estacion
cobertura global: 68.0 %

cobertura por estacion (% , ascendente):

estacion	
N03	25.0
NTE	38.9
NE3	49.8
N02	59.0
NE	63.4
NE2	68.6
NO	69.2
SE2	70.1
S0	73.8
SE	79.4
SUR	81.8
NTE2	82.3
S02	84.9
CE	86.0
SE3	88.2

estaciones con cobertura >= 85%: 2 de 15

Solo 2 de 15 estaciones (CE, SE3) libran el 85 % (S02 se queda en 84.9); la peor (N03) está en 25 %. El filtro por estación dejaría un análisis de dos series con huecos residuales de todos modos — no resuelve el problema, lo esconde en las estaciones que sobrevivieron el corte.

```
fig, ax = plt.subplots(figsize=(5.5, 3.2))
colores = [ROJO if v < 85 else AZUL for v in cobertura.values]
ax.barh(cobertura.index, cobertura.values, color=colores)
ax.axvline(85, color=GRIS, linestyle="--", linewidth=1, label="umbral 85%")
ax.set_xlabel("cobertura (%)")
ax.legend(fontsize=7, loc="lower right")
fig.tight_layout()
fig.savefig(FIGS / "serie_cobertura_estaciones.pdf")
plt.show()
```

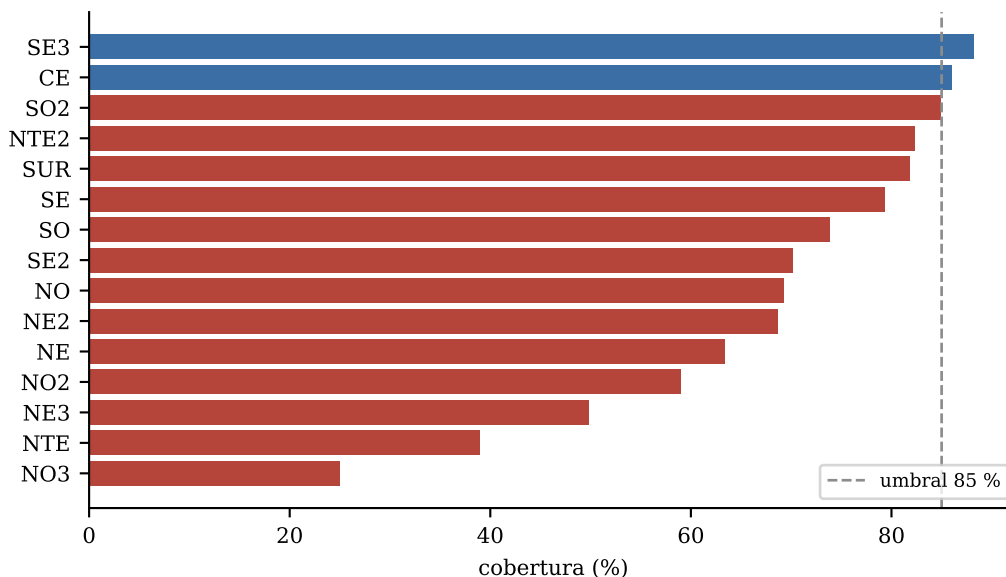


Figura 1: Cobertura de dia-estacion valido (factores + excede_anio_5) por estacion, sobre el calendario 2021-01-01 a 2025-12-31.

Dependencia transversal: la violación de independencia que no está documentada

Antes de promediar hay que confirmar que promediar tiene sentido: si las estaciones fueran independientes el mismo día, un promedio diario perdería la mitad de la señal en ruido de medición. Se calcula la correlación de Pearson entre estaciones, día por día, para cada una de las cuatro series.

```

VARS = FACTORES + ["excede_anio_5"]
filas = []
for col in VARS:
    piv = m.pivot(index="fecha", columns="estacion", values=col)
    corr = piv.corr()
    vals = corr.where(~np.eye(len(corr), dtype=bool)).stack()
    filas.append({"variable": col, "media": vals.mean(), "maximo": vals.max(),
                  "minimo": vals.min()})
transversal = pd.DataFrame(filas).set_index("variable")
print("correlacion entre estaciones, mismo dia (pares estacion-estacion):")
print(transversal.round(3).to_string())

```

correlacion entre estaciones, mismo dia (pares estacion-estacion):

	media	maximo	minimo
variable			
combustion_primaria	0.713	0.892	0.336
ventilacion	0.503	0.797	-0.156
forzamiento_fotoquimico	0.852	0.971	0.364
excede_anio_5	0.530	0.771	0.203

La correlación media entre estaciones va de 0.50 (ventilación) a 0.85 (forzamiento fotoquímico), y ningún par cae cerca de cero. Esto es exactamente lo que exige la física: la temperatura y la radiación solar (forzamiento fotoquímico) son fenómenos de escala regional y por eso su correlación entre estaciones es la más alta y la más uniforme (0.36 a 0.97); el viento (ventilación) depende de la topografía local del valle y por eso su rango es el más amplio, incluyendo un par con correlación ligeramente negativa (-0.16). Es la violación de independencia más grande del conjunto de datos: cualquier modelo que trate cada fila día-estación como una observación independiente —incluyendo la regresión logística y el discriminante de #55— está subestimando sus errores estándar. No se corrige aquí; se deja documentada para que #55 la considere.

Decisión: se agrega a nivel área metropolitana, promediando las estaciones que reportan cada día, en vez de analizar estaciones por separado. Dos razones, no una: la cobertura por estación es insuficiente (85 % lo libran solo 2 de 15) y la correlación entre estaciones el mismo día es lo bastante alta para que promediar sea agregación razonable y no una mezcla de señales independientes.

2. Construcción de la serie de área metropolitana

```
area = m.groupby("fecha").agg(
    combustion_primaria=("combustion_primaria", "mean"),
    ventilacion=("ventilacion", "mean"),
    forzamiento_fotoquimico=("forzamiento_fotoquimico", "mean"),
    frac_excede_anio_5=("excede_anio_5", "mean"),
    n_estaciones=("estacion", "nunique"),
).reset_index()
print(f"dias con al menos 1 estacion reportando: {len(area):,} de
    ↪ {len(full_days):,}")
print(area.n_estaciones.describe().round(1).to_string())
```

```
dias con al menos 1 estacion reportando: 1,825 de 1,826
count      1825.0
mean         10.2
std           2.8
min           1.0
25%           8.0
50%          11.0
75%          12.0
max          15.0
```

frac_excede_anio_5 es la fracción de estaciones que reportan (no de las 15) que exceden el umbral ese día; no es la indicadora binaria original, es su promedio de área. Con eso se reindexa contra el calendario completo, dejando los huecos explícitos como NaN.

```
AVARS = FACTORES + ["frac_excede_anio_5"]
area_full = area.set_index("fecha")[AVARS].reindex(full_days)
area_full.index.name = "fecha"
```

```

huecos = area_full["combustion_primaria"].isna()
print(f"dias faltantes tras reindexar (0 estaciones reportando ese dia): "
      f"{huecos.sum()} de {len(full_days)}"
      ↪  ({100*huecos.sum()/len(full_days):.3f}%)")
print("fecha(s) faltante(s):", list(full_days[huecos].date))

```

```

dias faltantes tras reindexar (0 estaciones reportando ese dia): 1 de 1826 (0.055 %)
fecha(s) faltante(s): [datetime.date(2021, 2, 16)]

```

Un solo día, 2021-02-16, se queda sin ninguna estación. Promediar resolvió la irregularidad casi por completo: de 68.0 % de cobertura día-estación a 99.94 % de cobertura del calendario diario de área. MSTL no admite NaN, así que ese único día se imputa — **solo porque STL/MSTL lo exige**, no porque el hueco deje de existir: queda documentado aquí y no se trata como si nunca hubiera pasado.

```

area_imp = area_full.interpolate(method="linear", limit_direction="both")
n_imputado = int(huecos.sum())
print(f"metodo de imputacion: interpolacion lineal entre el dia anterior y "
      f"el siguiente (n = 1 en cada lado)")
print(f"dias imputados: {n_imputado} de {len(full_days)}"
      ↪  ({100*n_imputado/len(full_days):.3f}%)")
print("\nserie final (area metropolitana, calendario regular):")
print(area_imp.describe().round(3).to_string())

```

```

metodo de imputacion: interpolacion lineal entre el dia anterior y el siguiente (n = 1 en cada
dias imputados: 1 de 1826 (0.055 %)

```

```

serie final (area metropolitana, calendario regular):

```

	combustion_primaria	ventilacion	forzamiento_fotoquimico	frac_excede_anio_5
count	1826.000	1826.000	1826.000	1826.000
mean	-0.125	-0.036	-0.011	0.314
std	0.907	0.664	1.112	0.353
min	-2.516	-2.578	-3.263	0.000
25%	-0.788	-0.498	-0.708	0.000
50%	-0.260	-0.060	0.111	0.143
75%	0.496	0.424	0.760	0.625
max	2.930	2.038	2.765	1.000

3. Descomposición MSTL: periodos 7 (semanal) y 365 (anual)

MSTL de statsmodels exige periodos enteros — 365.25 no es un entero válido y falla con `period must be a positive integer >= 2`. Se usa 365 en vez de 365.25: sobre 5 años de datos el corrimiento acumulado del ciclo anual es de aproximadamente un día, despreciable frente al horizonte de la serie. Se documenta la desviación en vez de forzar el valor pedido.

```
def ajusta_mstl(serie, periodos=(7, 365)):
    y = serie.asfreq("D")
    return MSTL(y, periods=periodos).fit()
```

```
modelos = {col: ajusta_mstl(area_imp[col]) for col in AVARS}
for col, res in modelos.items():
    amp_semanal = res.seasonal.iloc[:, 0].std()
    amp_anual = res.seasonal.iloc[:, 1].std()
    print(f"{col:26s} sd(estacional semanal)={amp_semanal:.4f}  "
          f"sd(estacional anual)={amp_anual:.4f}")
```

combustion_primaria	sd(estacional semanal)=0.2857	sd(estacional anual)=0.6389
ventilacion	sd(estacional semanal)=0.1634	sd(estacional anual)=0.4788
forzamiento_fotoquimico	sd(estacional semanal)=0.2342	sd(estacional anual)=0.8471
frac_excede_anio_5	sd(estacional semanal)=0.0888	sd(estacional anual)=0.2411

```
def grafica_mstl(res, titulo, archivo):
    componentes = [("observado", res.observado), ("tendencia", res.trend),
                   ("estacional semanal", res.seasonal.iloc[:, 0]),
                   ("estacional anual", res.seasonal.iloc[:, 1]),
                   ("residual", res.resid)]
    fig, axs = plt.subplots(len(componentes), 1, figsize=(6.5, 7.5), sharex=True)
    for ax, (nombre, serie) in zip(axs, componentes):
        ax.plot(serie.index, serie.values, color=AZUL, linewidth=0.6)
        ax.set_ylabel(nombre, fontsize=7)
        ax.tick_params(labelsize=6)
    axs[-1].set_xlabel("fecha")
    fig.suptitle(titulo, fontsize=9)
    fig.tight_layout()
    fig.savefig(FIGS / archivo)
    return fig
```

```
grafica_mstl(modelos["combustion_primaria"], "Combustion primaria",
             "mstl_combustion_primaria.pdf")
plt.show()
```

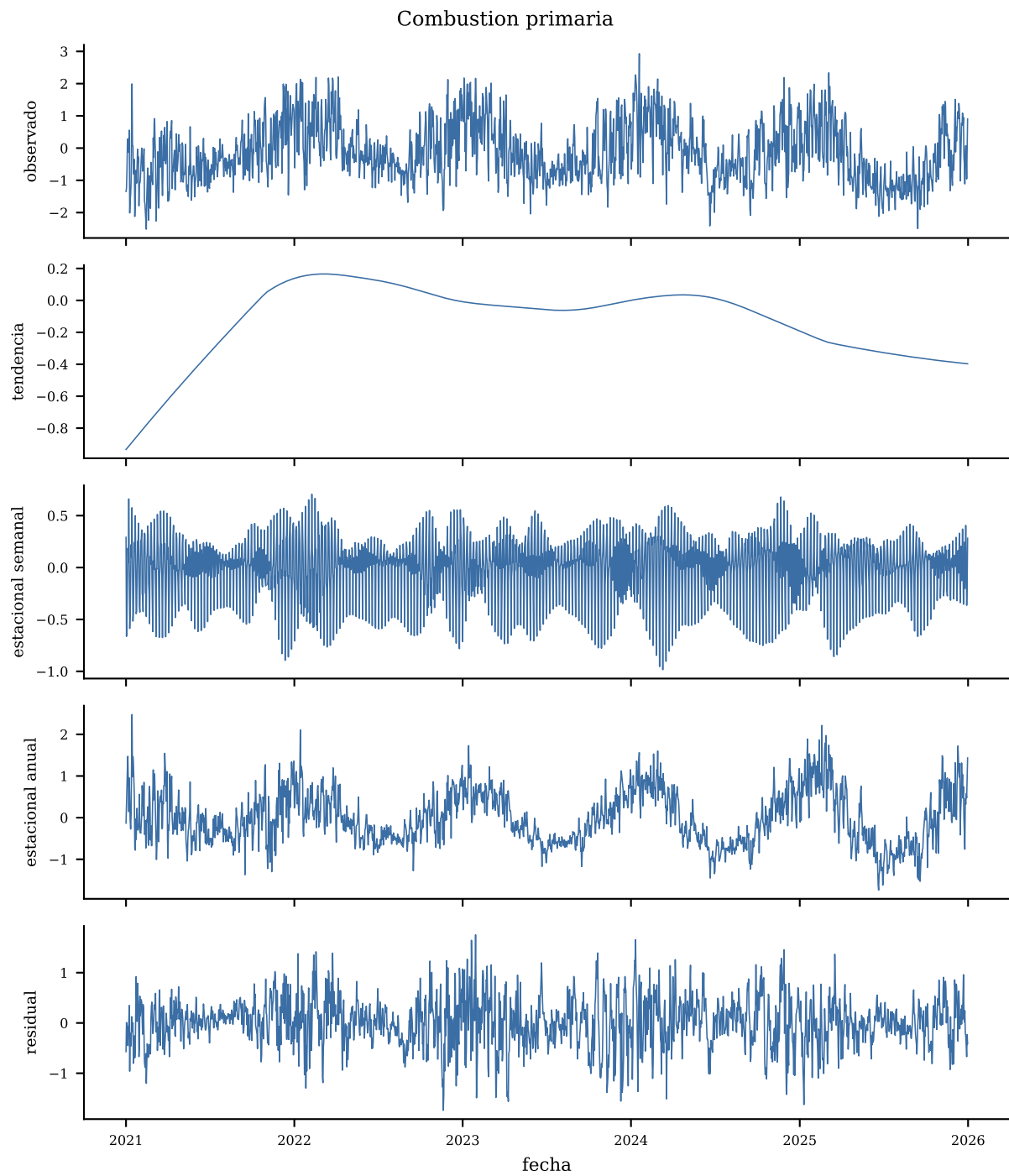



Figura 2: Descomposicion MSTL (periodos 7 y 365) de combustion primaria, promedio de area metropolitana.

```
grafica_mstl(modelos["ventilacion"], "Ventilacion", "mstl_ventilacion.pdf")
plt.show()
```

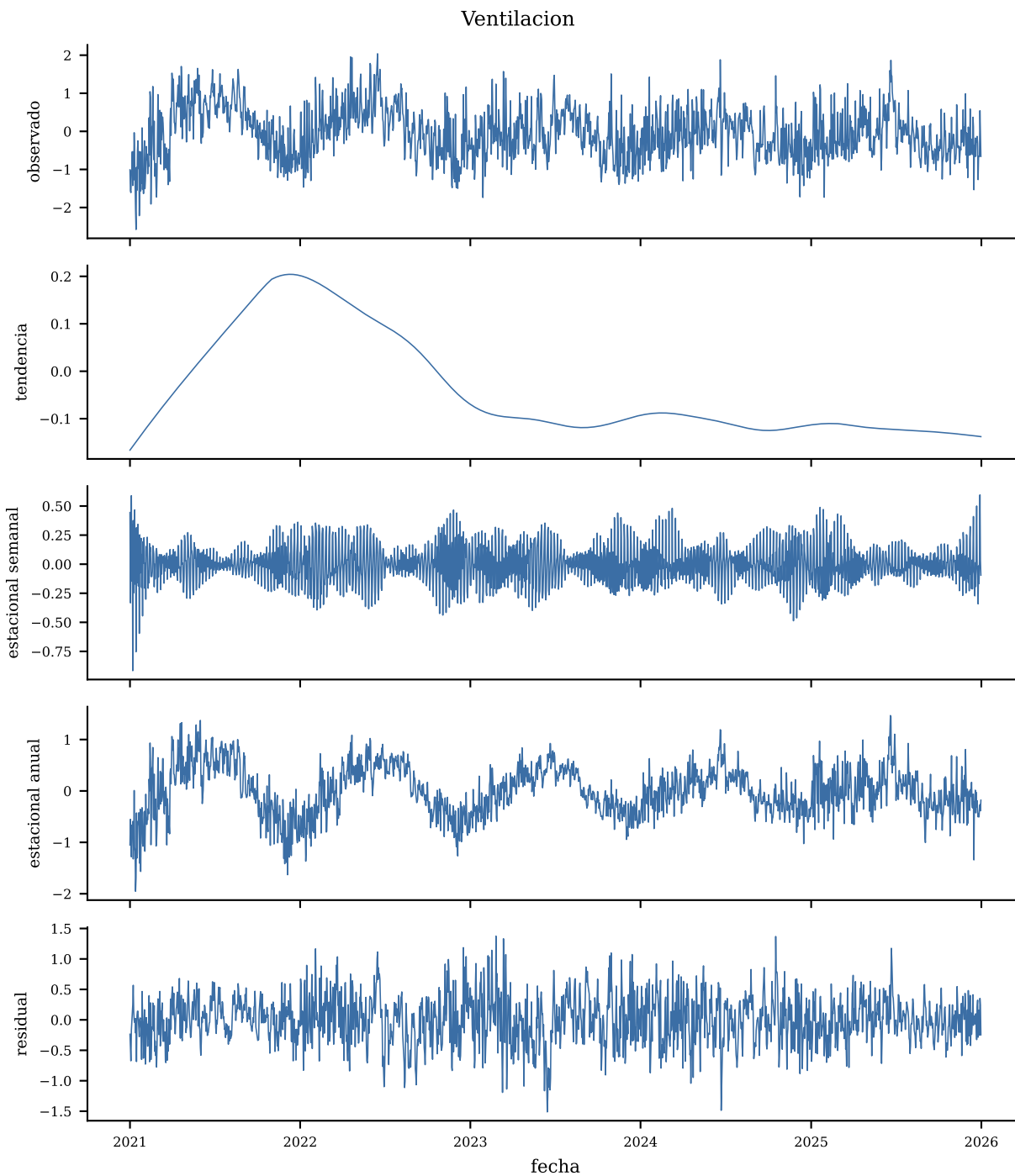


Figura 3: Descomposicion MSTL (periodos 7 y 365) de ventilacion, promedio de area metropolitana.

```
grafica_mstl(modelos["forzamiento_fotoquimico"], "Forzamiento fotoquimico",
             "mstl_forzamiento_fotoquimico.pdf")
plt.show()
```

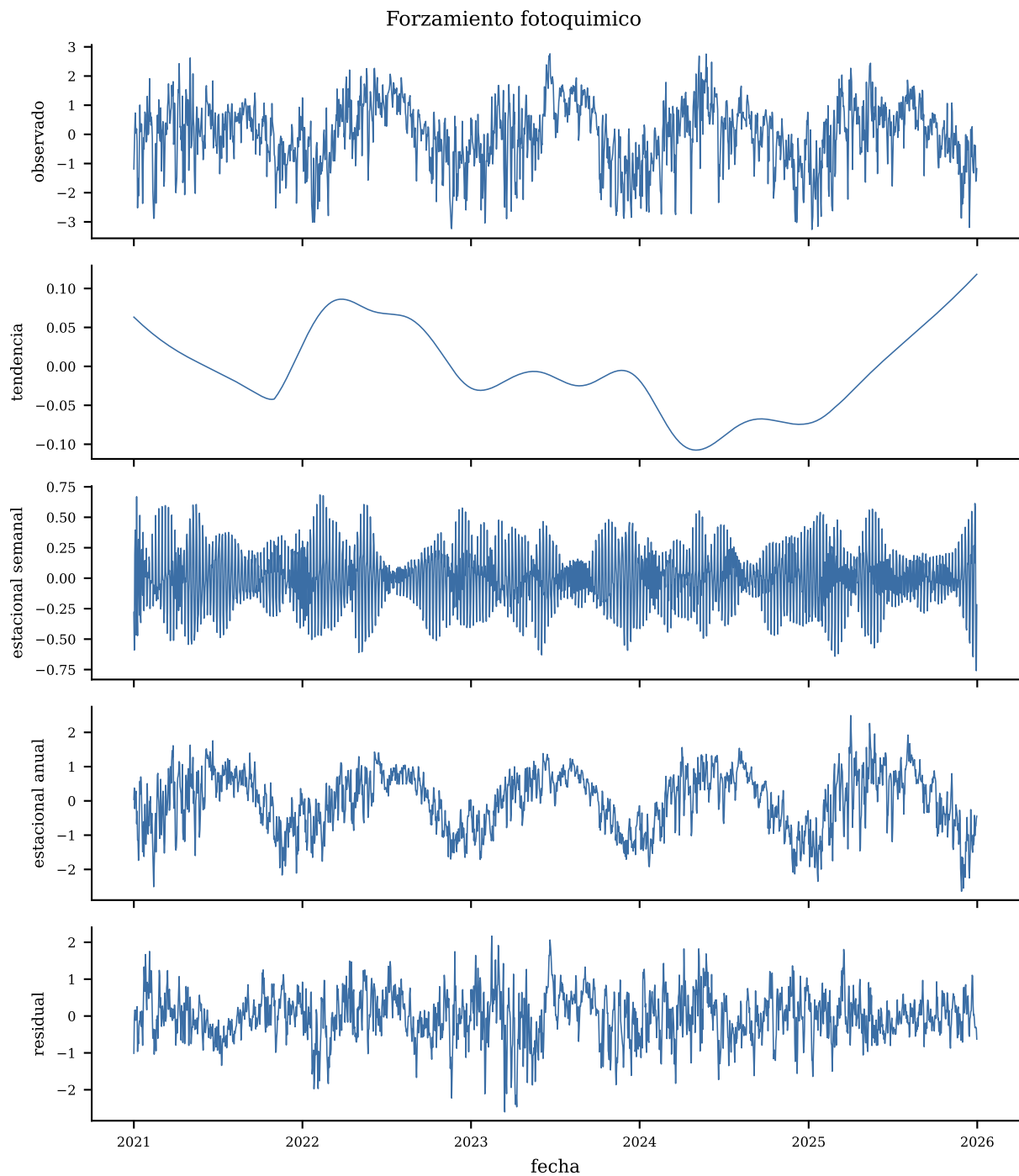


Figura 4: Descomposicion MSTL (periodos 7 y 365) de forzamiento fotoquimico, promedio de area metropolitana.

```
grafica_mstl(modelos["frac_excede_anio_5"], "Fraccion que excede (excede_anio_5)",
             "mstl_frac_excede_anio_5.pdf")
plt.show()
```

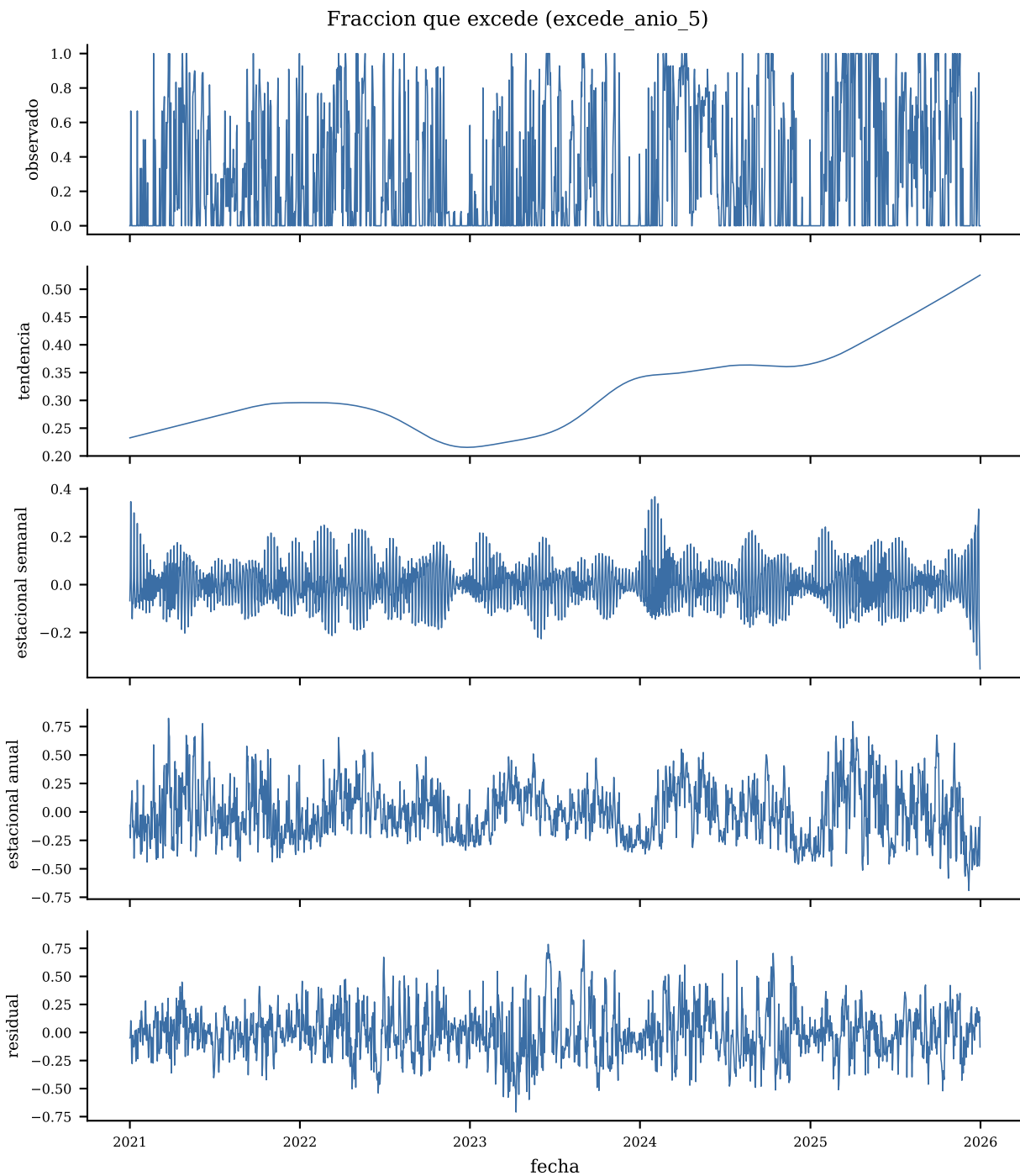


Figura 5: Descomposicion MSTL (periodos 7 y 365) de la fraccion de estaciones que exceden (`excede_anio_5`), promedio de area metropolitana.

La amplitud semanal es mayor en combustión primaria y menor en la fracción que excede: el ciclo entre-semana/fin-de-semana se ve con más claridad en los precursores (tráfico, industria) que en el resultado binario umbralizado, que diluye ese patrón al depender de varios factores a la vez.

4. ACF y PACF hasta el lag 40: serie cruda vs residual MSTL

Se calculan juntas dos versiones — sobre la serie cruda de área metropolitana y sobre el residual de la descomposición MSTL — porque comparar ambas es lo que muestra cuánta autocorrelación explica la estacionalidad semanal/anual y cuánta le sobrevive.

```
n = len(area_imp)
banda = 1.96 / np.sqrt(n)
print(f"n = {n}    banda +-1.96/sqrt(n) = +-{banda:.4f}")

def calcula_diagnostico(serie, nlags=40):
    a = acf(serie, nlags=nlags, fft=True)
    p = pacf(serie, nlags=nlags, method="ywmm")
    return a, p

diagnosticos = {}
for col in AVARS:
    cruda = area_imp[col].asfreq("D")
    residual = modelos[col].resid
    diagnosticos[col] = {"cruda": calcula_diagnostico(cruda),
                        "residual": calcula_diagnostico(residual)}
```

```
n = 1826    banda +-1.96/sqrt(n) = +-0.0459
```

```
def grafica_acf_pacf(col, titulo, archivo):
    fig, axs = plt.subplots(2, 2, figsize=(6.5, 4.8), sharex=True)
    for fila, version in enumerate(["cruda", "residual"]):
        a, p = diagnosticos[col][version]
        lags = np.arange(len(a))
        for ax, valores, etiqueta in [(axs[fila, 0], a, "ACF"), (axs[fila, 1], p,
            ↪ "PACF")]:
            ax.bar(lags[1:], valores[1:], color=AZUL, width=0.7)
            ax.axhline(banda, color=ROJO, linestyle="--", linewidth=0.8)
            ax.axhline(-banda, color=ROJO, linestyle="--", linewidth=0.8)
            ax.axhline(0, color="black", linewidth=0.5)
            ax.set_title(f"{etiqueta} ({version})", fontsize=7)
            ax.tick_params(labelsize=6)
    axs[1, 0].set_xlabel("lag"); axs[1, 1].set_xlabel("lag")
    fig.suptitle(titulo, fontsize=9)
    fig.tight_layout()
    fig.savefig(FIGS / archivo)
    return fig

grafica_acf_pacf("combustion_primaria", "Combustion primaria",
                 "acf_pacf_combustion_primaria.pdf")
plt.show()
```

Combustion primaria

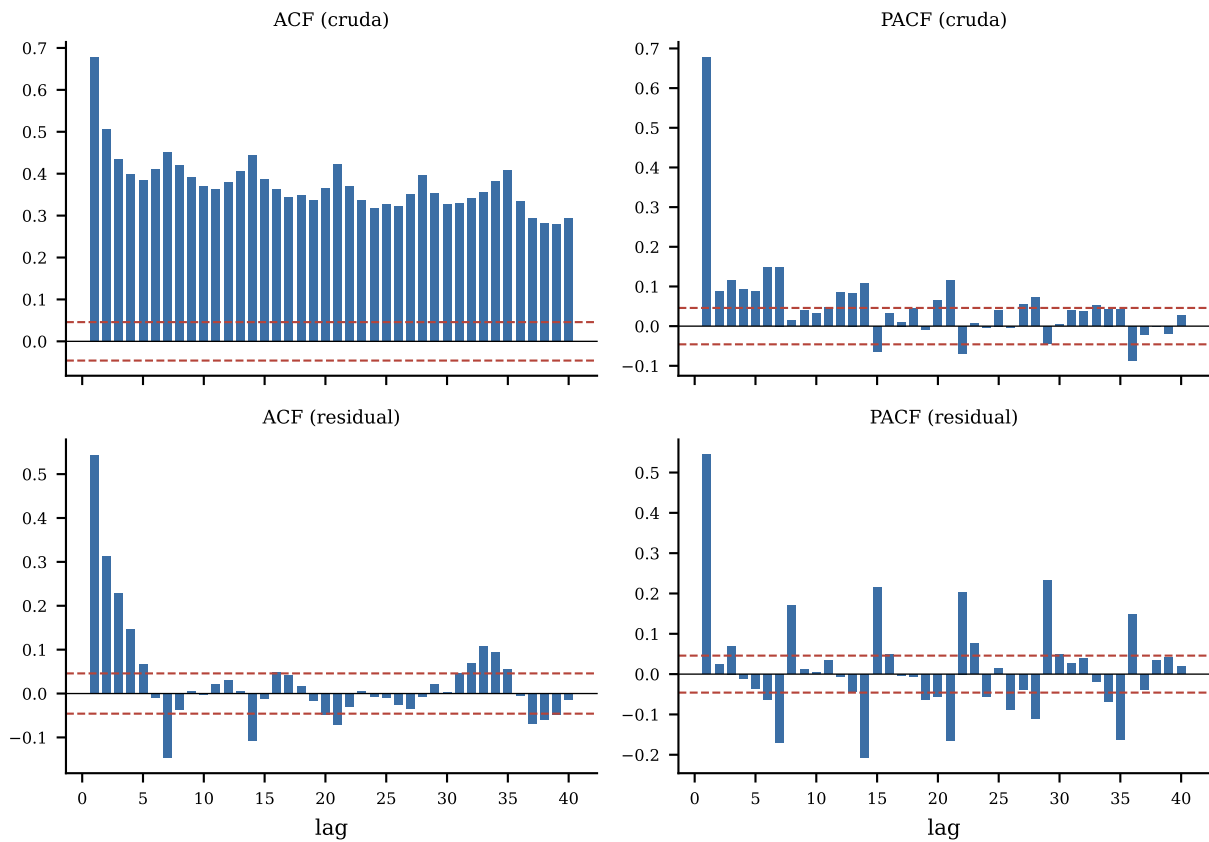


Figura 6: ACF y PACF hasta el lag 40 de combustion primaria: serie cruda (arriba) vs residual MSTL (abajo), con banda $\pm 1.96/\sqrt{n}$.

```
grafica_acf_pacf("ventilacion", "Ventilacion", "acf_pacf_ventilacion.pdf")
plt.show()
```

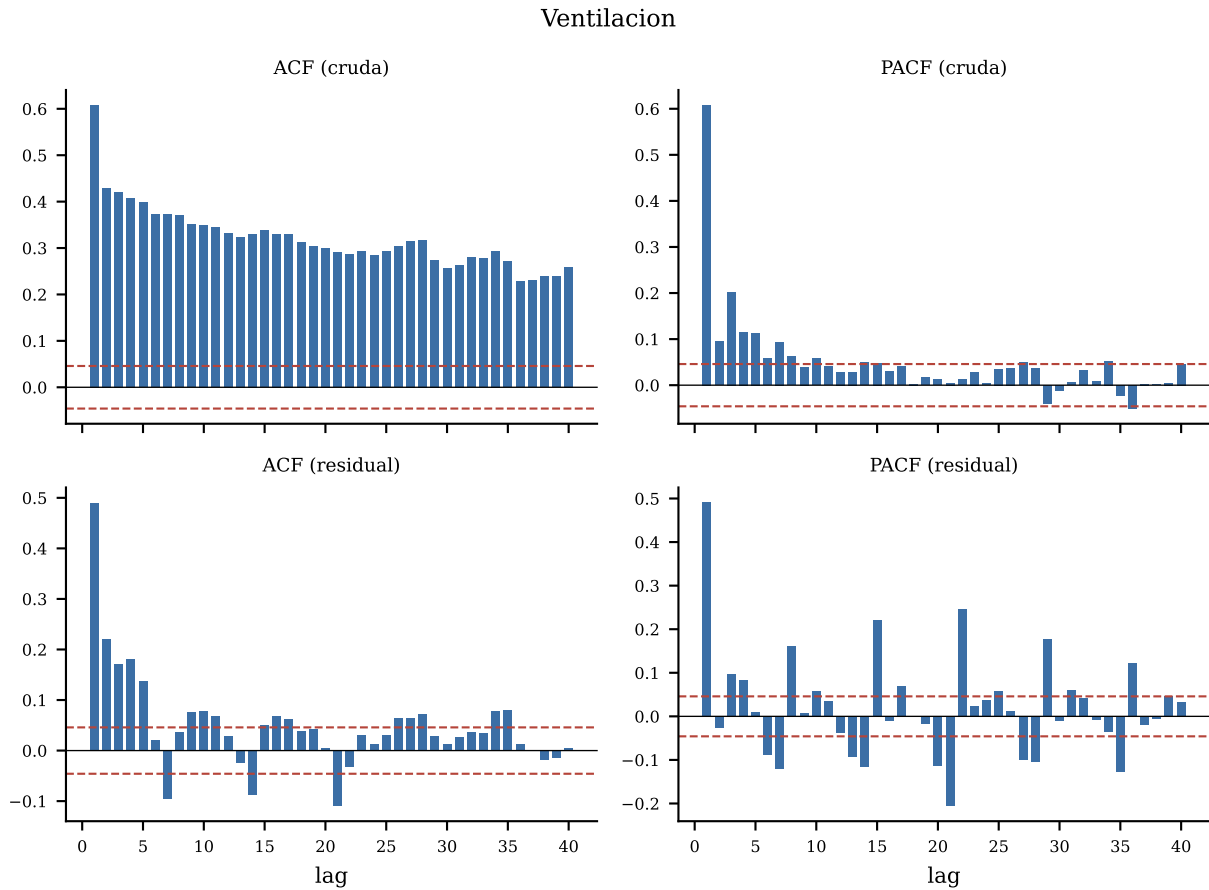


Figura 7: ACF y PACF hasta el lag 40 de ventilacion: serie cruda (arriba) vs residual MSTL (abajo), con banda $\pm 1.96/\sqrt{n}$.

```
grafica_acf_pacf("forzamiento_fotoquimico", "Forzamiento fotoquimico",
                 "acf_pacf_forzamiento_fotoquimico.pdf")
plt.show()
```

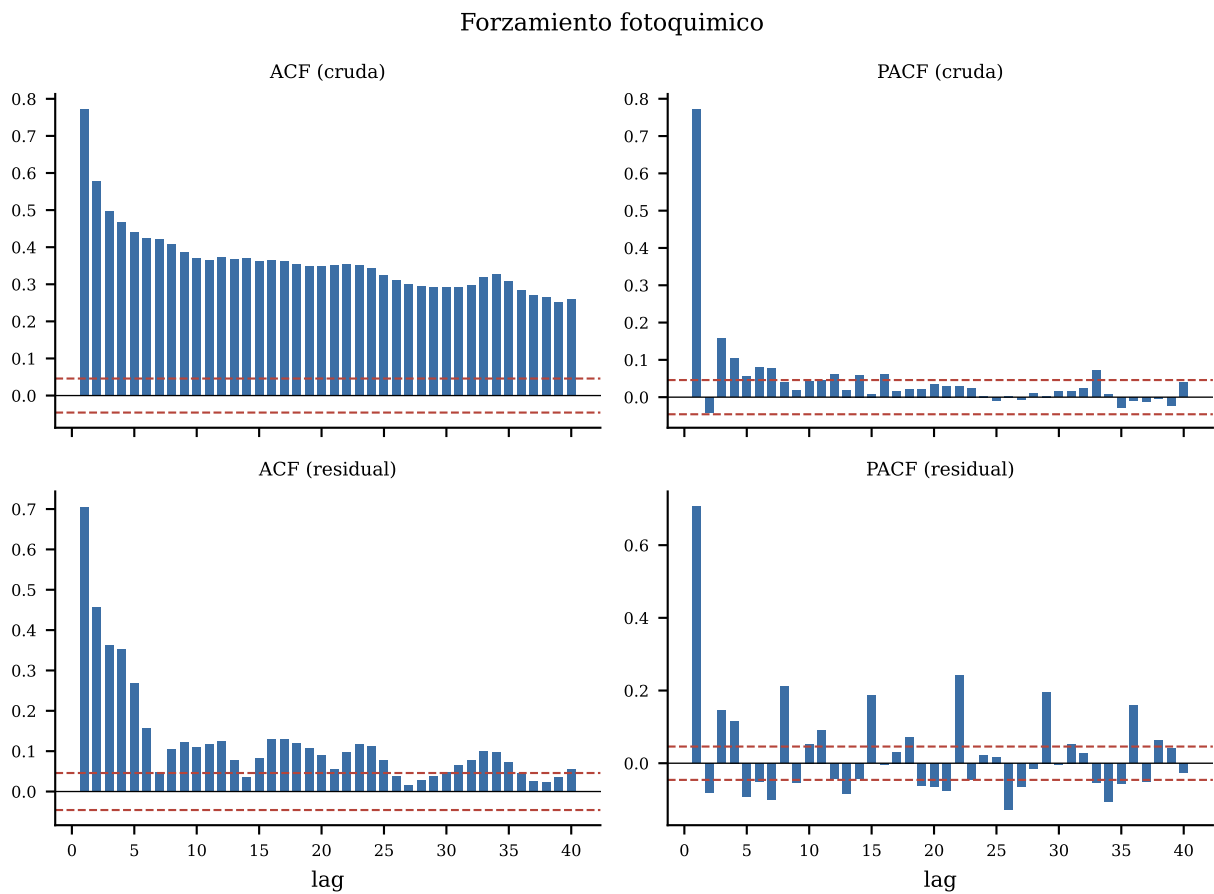


Figura 8: ACF y PACF hasta el lag 40 de forzamiento fotoquímico: serie cruda (arriba) vs residual MSTL (abajo), con banda $\pm 1.96/\sqrt{n}$.

```
grafica_acf_pacf("frac_excede_anio_5", "Fraccion que excede (excede_anio_5)",
                 "acf_pacf_frac_excede_anio_5.pdf")
plt.show()
```

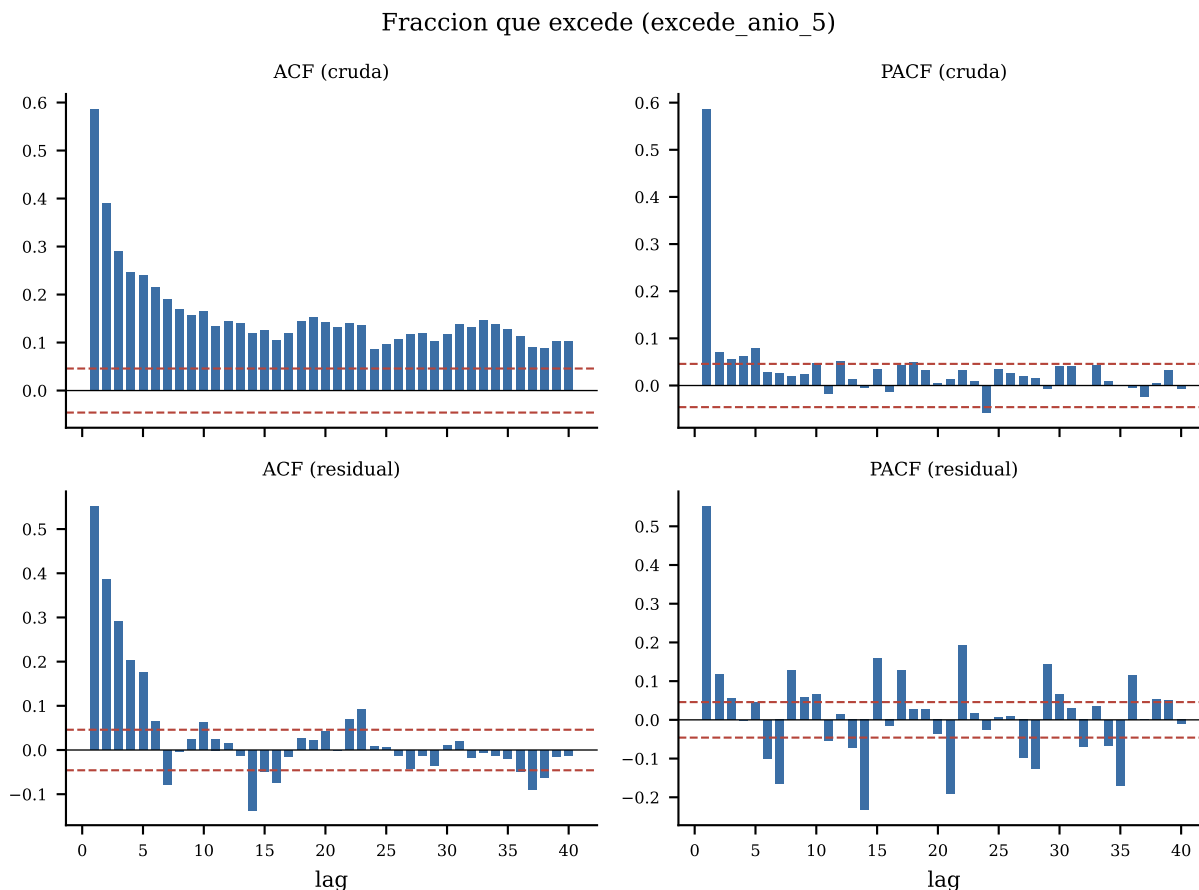



Figura 9: ACF y PACF hasta el lag 40 de la fraccion que excede: serie cruda (arriba) vs residual MSTL (abajo), con banda $\pm 1.96/\sqrt{n}$.

```
resumen = []
for col in AVARS:
    for version in ["cruda", "residual"]:
        a, _ = diagnosticos[col][version]
        fuera = int((np.abs(a[1:]) > banda).sum())
        resumen.append({"variable": col, "version": version, "acf_lag1": a[1],
                        "acf_lag7": a[7], "lags_fuera_de_banda_1_40": fuera})
resumen = pd.DataFrame(resumen)
print(resumen.round(3).to_string(index=False))
```

	variable	version	acf_lag1	acf_lag7	lags_fuera_de_banda_1_40
	combustion_primaria	cruda	0.678	0.452	40
	combustion_primaria	residual	0.544	-0.147	18
	ventilacion	cruda	0.608	0.374	40
	ventilacion	residual	0.491	-0.095	19
	forzamiento_fotoquimico	cruda	0.772	0.421	40
	forzamiento_fotoquimico	residual	0.706	0.049	32
	frac_excede_anio_5	cruda	0.586	0.191	40

5. Ljung-Box en lags 7, 14 y 30 (ambas versiones)

```

filas_lb = []
for col in AVARS:
    for version, serie in [("cruda", area_imp[col].asfreq("D")), ("residual",
        ↪ modelos[col].resid)]:
        lb = acorr_ljungbox(serie, lags=[7, 14, 30], return_df=True)
        for lag, fila in lb.iterrows():
            filas_lb.append({"variable": col, "version": version, "lag": lag,
                ↪ "estadistico": fila["lb_stat"], "p_valor":
                ↪ fila["lb_pvalue"]})
tabla_lb = pd.DataFrame(filas_lb)
print(tabla_lb.round(4).to_string(index=False))

```

	variable	version	lag	estadistico	p_valor
	combustion_primaria	cruda	7	2906.7443	0.0
	combustion_primaria	cruda	14	4937.8457	0.0
	combustion_primaria	cruda	30	8684.1007	0.0
	combustion_primaria	residual	7	904.2912	0.0
	combustion_primaria	residual	14	930.9235	0.0
	combustion_primaria	residual	30	960.7015	0.0
	ventilacion	cruda	7	2441.8587	0.0
	ventilacion	cruda	14	3957.7191	0.0
	ventilacion	cruda	30	6669.1998	0.0
	ventilacion	residual	7	695.0988	0.0
	ventilacion	residual	14	744.9704	0.0
	ventilacion	residual	30	825.9621	0.0
	forzamiento_fotoquimico	cruda	7	3575.1573	0.0
	forzamiento_fotoquimico	cruda	14	5414.3877	0.0
	forzamiento_fotoquimico	cruda	30	8765.6628	0.0
	forzamiento_fotoquimico	residual	7	1947.7100	0.0
	forzamiento_fotoquimico	residual	14	2086.7402	0.0
	forzamiento_fotoquimico	residual	30	2321.8518	0.0
	frac_excede_anio_5	cruda	7	1430.2834	0.0
	frac_excede_anio_5	cruda	14	1712.0309	0.0
	frac_excede_anio_5	cruda	30	2161.7593	0.0
	frac_excede_anio_5	residual	7	1132.0008	0.0
	frac_excede_anio_5	residual	14	1176.2564	0.0
	frac_excede_anio_5	residual	30	1226.6788	0.0

Con $n = 1,826$ la prueba rechaza casi por construcción, igual que ya se documentó para Bartlett en 08_analisis_factorial.pdf: el estadístico de Ljung-Box crece con n , y con más de mil observaciones diarias cualquier autocorrelación no trivial, por pequeña que sea, produce un

p-valor indistinguible de cero. Los p-valores de esta tabla son todos < 0.001 y no discriminan nada — rechazan tanto la serie cruda (donde se espera) como el residual MSTL (donde la estacionalidad ya se quitó). Lo que sí informa es la **magnitud de la ACF**: cae de 0.19–0.45 a valores entre -0.15 y 0.05 en el lag 7 tras MSTL (la estacionalidad semanal casi desaparece), pero el lag 1 se queda entre 0.49 y 0.71 en las cuatro series incluso después de la descomposición — esa persistencia de un día para otro no es estacionalidad, es autocorrelación genuina de corto plazo que MSTL no está diseñado para quitar.

6. Rezagos: contra la fecha calendario, no `shift()` de fila

No se construyen rezagos como predictor en esta notebook — es exploración descriptiva, no modelado — pero si #55 los necesita a partir de aquí, la regla es: se calculan sobre `area_imp`, que ya tiene índice `DatetimeIndex` diario regular (sin huecos, tras la imputación de la sección 2), así que un `.shift()` sobre esa tabla sí corresponde a “un día calendario antes”. Nunca sobre `m` (la tabla día-estación original): con 32 % de días ausentes por estación, la fila anterior de un `.sort_values("fecha")` puede estar a una semana de distancia y no a un día.

7. Criterios de aceptación

OK	El renombrado F1/F2/F3 corresponde a <code>af_cargas.csv</code> ($r > 0.8$)
OK	La cobertura por estacion se reporta y se decide con base en ella
OK	La correlacion transversal entre estaciones se documenta (media, maximo)
OK	El calendario se reindexa completo y los huecos quedan explicitos antes de imputar
OK	MSTL corre con periodos enteros (7, 365), documentando la diferencia con 365.25
OK	ACF/PACF hasta el lag 40 se calculan en las dos versiones (cruda y residual)
OK	Ljung-Box se reporta en lags 7/14/30 para ambas versiones

8. Qué queda para el reporte

Cuatro hallazgos con lugar en el reporte: (1) solo 2 de 15 estaciones superan 85 % de cobertura día-estación, pero promediar a nivel área metropolitana y reindexar contra el calendario completo recupera 99.94 % de cobertura (un solo día sin ninguna estación, 2021-02-16); (2) la correlación entre estaciones el mismo día es alta y no está documentada en ningún otro análisis — media de 0.50 (ventilación) a 0.85 (forzamiento fotoquímico), con un mínimo ligeramente negativo solo en ventilación — y es la violación de independencia más grande del conjunto de datos, relevante para cualquier modelo que trate las filas día-estación como independientes; (3) MSTL separa un ciclo semanal real y de amplitud dispar entre variables (más fuerte en combustión primaria, más débil en la fracción que excede el umbral); y (4) la autocorrelación de lag 1 sobrevive a la descomposición MSTL en las cuatro series (0.49–0.71 en el residual), mientras que la de lag 7 casi desaparece — evidencia de que la dependencia temporal día-a-día de este panel no es un artefacto de estacionalidad y debe considerarse en cualquier modelo que suponga observaciones independientes.